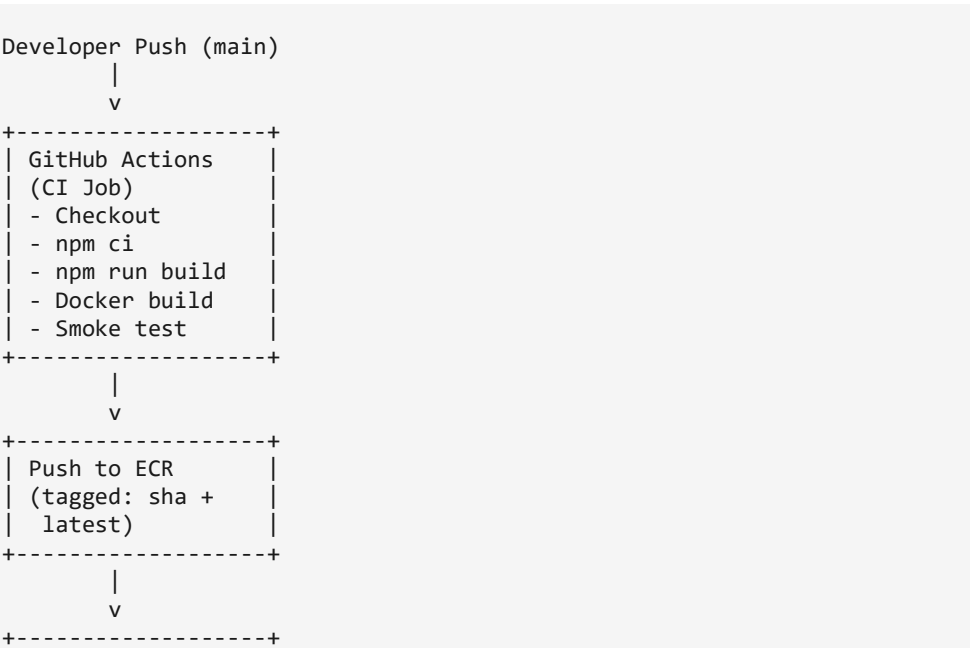
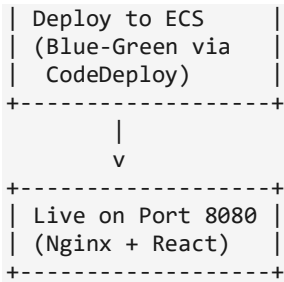


CI/CD Automation Runbook

Document Info	Details
Project	CoreUI React Admin Template
Repository	sanket-tg/coreui-free-react-admin-template-main
Pipeline	GitHub Actions → AWS ECR → ECS (Blue-Green via CodeDeploy)
Last Updated	July 2026
Author	Sanket Patil
Reviewer	Prajwal Gaikwad

1. Architecture Overview





2. Pipeline Triggers

Event	Branch	Action
Push	main	Full pipeline: CI → Push → Deploy
Pull Request	main	CI only (build + smoke test, no deploy)
Manual	Any	Triggered via <code>workflow_dispatch</code> in GitHub UI

3. Pipeline Jobs Breakdown

Job 1: CI - Build & Test

Step	Description
Checkout	Pulls latest code from the repository
Set image tag	Uses short git SHA as the Docker image tag
Setup Node.js 22	Installs Node.js with npm cache
Install dependencies	<code>npm ci</code> (clean install from lockfile)

Build React app	<code>npm run build</code> produces production assets
AWS credential config	Assumes IAM role via OIDC (no stored keys)
ECR Login	Authenticates Docker to push/pull from ECR
Docker Build	Multi-stage build (Node build + Nginx serve)
Smoke Test	Starts container, waits 8s, curls port 8080

Job 2: Push Image to ECR

- Runs only on `push` events (not PRs)
- Builds and pushes image with two tags: `<sha>` and `latest`
- Uses GitHub Actions build cache (`type=gha`) for speed

Job 3: Deploy to ECS (Blue-Green)

- Runs only when pushed to `main` branch
- Downloads current ECS task definition
- Renders new task definition with updated image URI
- Deploys via AWS CodeDeploy blue-green strategy
- Waits for service stability before marking success
- Generates deployment summary in GitHub Actions UI

4. AWS Infrastructure Components

Component	Purpose
Amazon ECR	Docker image registry (stores built images)
Amazon ECS (Fargate)	Runs containerized app as a service
AWS CodeDeploy	Manages blue-green deployment switching

Application Load Balancer	Routes traffic to active (blue/green) target group
IAM OIDC Provider	Allows GitHub Actions to assume AWS roles without access keys

Key Configuration Variables (GitHub Settings)

Type	Name	Description
Secret	AWS_ROLE_ARN	IAM role ARN for OIDC-based auth
Variable	AWS_REGION	AWS region (e.g., ap-south-1)
Variable	ECR_REPOSITORY	ECR repository name
Variable	ECS_CLUSTER	ECS cluster name
Variable	ECS_SERVICE	ECS service name
Variable	ECS_TASK_DEFINITION	Task definition family name
Variable	CODEDEPLOY_APPLICATION	CodeDeploy application name
Variable	CODEDEPLOY_DEPLOYMENT_GROUP	CodeDeploy deployment group

5. Docker Build Strategy

Multi-stage Dockerfile for minimal production image:

Stage	Base Image	Purpose
-------	------------	---------

Build	<code>node:20-alpine</code>	Install deps, compile React app
Serve	<code>nginx:stable-alpine</code>	Serve static assets on port 8080

Nginx is configured to:

- Serve from `/usr/share/nginx/html`
- Handle client-side routing (SPA fallback to `index.html`)
- Cache static assets (CSS, JS, images) for 1 year

6. Deployment Flow (Blue-Green)

Step	What Happens
1	New task definition registered with updated image
2	CodeDeploy launches new (green) task set
3	Health checks pass on green target group
4	Traffic shifts from blue to green
5	Old (blue) task set terminated after stabilization

Note: If the green deployment fails health checks, CodeDeploy automatically rolls back to the blue (previous) version.

7. Troubleshooting Quick Reference

Issue	Check
Pipeline not triggering	Verify push is to <code>main</code> branch; check Actions tab for workflow runs

AWS credential error	Verify <code>AWS_ROLE_ARN</code> secret and OIDC trust policy
Docker build fails	Check <code>npm ci</code> / <code>npm run build</code> logs; verify Node version
Smoke test fails	Container may need more startup time; check Nginx config
ECR push denied	Verify IAM role has <code>ecr:PutImage</code> permissions
ECS deploy stuck	Check ECS service events; verify task health checks and ALB target group health
Rollback triggered	Check CodeDeploy deployment logs for failure reason
Site not updating	Hard refresh browser (Ctrl+Shift+R); check if deploy job completed

8. Manual Operations

Trigger Pipeline Manually

Go to GitHub → Actions → Select workflow → Click "Run workflow"

Rollback to Previous Version

Option A: Revert commit on `main` and push (triggers fresh deploy)

Option B: In AWS CodeDeploy console, select deployment → "Stop and roll back"

View Deployment Status

- GitHub: Actions tab → latest workflow run → Deploy job summary
- AWS: ECS Console → Cluster → Service → Deployments tab
- AWS: CodeDeploy Console → Deployments